

Hands on 5

Implement services for managing Country

OrmLearnApplication.java

```
package com.cognizant.orm_learn;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class OrmLearnApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(OrmLearnApplication.class, args);
```

```
    }
```

```
}
```

CountryTestRunner.java

```
package com.cognizant.orm_learn;
```

```
import com.cognizant.orm_learn.entity.Country;
```

```
import com.cognizant.orm_learn.service.CountryService;
```

```
import org.springframework.boot.CommandLineRunner;
```

```
import org.springframework.stereotype.Component;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import java.util.List;
```

```
import java.util.Optional;
```

@Component

```
public class CountryTestRunner implements CommandLineRunner {
```

@Autowired

```
private CountryService countryService;
```

@Override

```
public void run(String... args) {
```

```
    // 1. Find by country code
```

```
    Optional<Country> india = countryService.findByCountryCode("IN");
```

```
    System.out.println("Find IN: " + india.map(Country::getCoName).orElse("Not  
found"));
```

```
    // 2. Add new country
```

```
    countryService.addCountry(new Country("XX", "Testland"));
```

```
    System.out.println("Added XX - Testland");
```

```
    // 3. Update country
```

```
    countryService.updateCountry(new Country("XX", "Testland Updated"));
```

```
    System.out.println("Updated XX");
```

```
    // 4. Delete country
```

```
    countryService.deleteCountry("XX");
```

```
    System.out.println("Deleted XX");
```

```
    // 5. Partial name search
```

```
    List<Country> matches = countryService.findByPartialName("stan");
```

```
    matches.forEach(c -> System.out.println(c.getCoCode() + " - " + c.getCoName()));
```

```
}  
}
```

CountryServiceImpl.java

```
package com.cognizant.orm_learn.service;  
  
import com.cognizant.orm_learn.entity.Country;  
import com.cognizant.orm_learn.repository.CountryRepository;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Service;  
  
import java.util.List;  
import java.util.Optional;  
  
@Service  
public class CountryServiceImpl implements CountryService {  
  
    @Autowired  
    private CountryRepository countryRepository;  
  
    @Override  
    public Optional<Country> findByCountryCode(String coCode) {  
        return countryRepository.findById(coCode);  
    }  
  
    @Override  
    public Country addCountry(Country country) {  
        return countryRepository.save(country);  
    }  
}
```

@Override

```
public Country updateCountry(Country country) {  
    return countryRepository.save(country);  
}
```

@Override

```
public void deleteCountry(String coCode) {  
    countryRepository.deleteById(coCode);  
}
```

@Override

```
public List<Country> findByPartialName(String partialName) {  
    return countryRepository.findByCoNameContainingIgnoreCase(partialName);  
}  
}
```

CountryService.java

```
package com.cognizant.orm_learn.service;
```

```
import com.cognizant.orm_learn.entity.Country;
```

```
import java.util.List;
```

```
import java.util.Optional;
```

```
public interface CountryService {
```

```
    Optional<Country> findByCountryCode(String coCode);
```

```
Country addCountry(Country country);
```

```
Country updateCountry(Country country);
```

```
void deleteCountry(String coCode);
```

```
List<Country> findByPartialName(String partialName);
```

```
}
```

CountryRepository.java

```
package com.cognizant.orm_learn.repository;
```

```
import com.cognizant.orm_learn.entity.Country;
```

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import java.util.List;
```

```
public interface CountryRepository extends JpaRepository<Country, String> {
```

```
List<Country> findByCoNameContainingIgnoreCase(String partialName);
```

```
}
```

Country.java

```
package com.cognizant.orm_learn.entity;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
@Table(name = "country")
```

```
public class Country {
```

@Id

@Column(name = "co_code", length = 2)

private String coCode;

@Column(name = "co_name")

private String coName;

public Country() {}

public Country(String coCode, String coName) {

this.coCode = coCode;

this.coName = coName;

}

public String getCoCode() { **return** coCode; }

public void setCoCode(String coCode) { **this**.coCode = coCode; }

public String getCoName() { **return** coName; }

public void setCoName(String coName) { **this**.coName = coName; }

}

```

2026-07-07T12:57:36.421+05:30 INFO 23752 --- [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-07-07T12:57:36.554+05:30 INFO 23752 --- [ restartedMain] org.hibernate.orm.jpa : HHH000540: Processing PersistenceUnitInfo [name: default]
2026-07-07T12:57:36.630+05:30 INFO 23752 --- [ restartedMain] org.hibernate.orm.core : HHH000001: Hibernate ORM core version 7.4.1.Final
2026-07-07T12:57:37.094+05:30 INFO 23752 --- [ restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-07-07T12:57:37.208+05:30 INFO 23752 --- [ restartedMain] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
Database JDBC URL [jdbc:mysql://localhost:3306/ormlearn]
Database driver: MySQL Connector/J
Database dialect: MySQLDialect
Database version: 8.0.37
Default catalog/schema: ormlearn/undefined
Autocommit mode: undefined/unknown
Isolation level: REPEATABLE_READ [default REPEATABLE_READ]
JDBC fetch size: none
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-07-07T12:57:37.917+05:30 INFO 23752 --- [ restartedMain] org.hibernate.orm.core : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' t
2026-07-07T12:57:37.958+05:30 INFO 23752 --- [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-07-07T12:57:38.046+05:30 INFO 23752 --- [ restartedMain] o.s.d.j.r.query.QueryEnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be used.
2026-07-07T12:57:38.317+05:30 INFO 23752 --- [ restartedMain] c.c.orm.learn.OrmLearnApplication : Started OrmLearnApplication in 3.548 seconds (process running for 3.964)
2026-07-07T12:57:38.388+05:30 DEBUG 23752 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Find IN: India
2026-07-07T12:57:38.427+05:30 DEBUG 23752 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T12:57:38.472+05:30 DEBUG 23752 --- [ restartedMain] org.hibernate.SQL : insert into country (co_name,co_code) values (?,?)
Hibernate: insert into country (co_name,co_code) values (?,?)
Added XX - Testland
2026-07-07T12:57:38.522+05:30 DEBUG 23752 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T12:57:38.531+05:30 DEBUG 23752 --- [ restartedMain] org.hibernate.SQL : update country set co_name=? where co_code=?
Hibernate: update country set co_name=? where co_code=?
Updated XX
2026-07-07T12:57:38.540+05:30 DEBUG 23752 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T12:57:38.546+05:30 DEBUG 23752 --- [ restartedMain] org.hibernate.SQL : delete from country where co_code=?
Hibernate: delete from country where co_code=?
Deleted XX
2026-07-07T12:57:39.076+05:30 DEBUG 23752 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) lik
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) like upper(?) escape '\\'
AF - Afghanistan
KZ - Kazakhstan
KG - Kyrgyzstan
PK - Pakistan
SH - Saint Helena, Ascension and Tristan da Cunha
TJ - Tajikistan
TM - Turkmenistan
UZ - Uzbekistan
2026-07-07T12:57:39.084+05:30 INFO 23752 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-07-07T12:57:39.085+05:30 INFO 23752 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown initiated...
2026-07-07T12:57:39.093+05:30 INFO 23752 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown completed.

```

Hands on 6

Find a country based on country code

CountryNotFoundException.java

java

package com.cognizant.orm_learn.service.exception;

```
public class CountryNotFoundException extends Exception {
```

```
    public CountryNotFoundException(String message) {
        super(message);
    }
```

```
    public CountryNotFoundException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

CountryService.java

java

package com.cognizant.orm_learn.service;

import com.cognizant.orm_learn.entity.Country;

import com.cognizant.orm_learn.service.exception.CountryNotFoundException;

import java.util.List;

import java.util.Optional;

public interface CountryService {

Optional<Country> findByCountryCode(String coCode);

Country findCountryByCode(String countryCode) throws CountryNotFoundException;

Country addCountry(Country country);

Country updateCountry(Country country);

void deleteCountry(String coCode);

List<Country> findByPartialName(String partialName);

}

CountryServiceImpl.java

java

package com.cognizant.orm_learn.service;


```
import com.cognizant.orm_learn.entity.Country;
import com.cognizant.orm_learn.repository.CountryRepository;
import com.cognizant.orm_learn.service.exception.CountryNotFoundException;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

```
import java.util.List;
import java.util.Optional;
```

```
@Service
```

```
public class CountryServiceImpl implements CountryService {
```

```
    @Autowired
```

```
    private CountryRepository countryRepository;
```

```
    @Override
```

```
    public Optional<Country> findByCountryCode(String coCode) {
        return countryRepository.findById(coCode);
    }
```

```
    @Transactional
```

```
    @Override
```

```
    public Country findCountryByCode(String countryCode) throws
CountryNotFoundException {
        Optional<Country> result = countryRepository.findById(countryCode);

        if (!result.isPresent()) {
```

```
        throw new CountryNotFoundException("Country not found for code: " +
countryCode);
    }
}
```

```
        Country country = result.get();
        return country;
    }
}
```

```
@Override
public Country addCountry(Country country) {
    return countryRepository.save(country);
}
}
```

```
@Override
public Country updateCountry(Country country) {
    return countryRepository.save(country);
}
}
```

```
@Override
public void deleteCountry(String coCode) {
    countryRepository.deleteById(coCode);
}
}
```

```
@Override
public List<Country> findByPartialName(String partialName) {
    return countryRepository.findByCoNameContainingIgnoreCase(partialName);
}
}
```

OrmLearnApplication.java

```
java

package com.cognizant.orm_learn;

import com.cognizant.orm_learn.entity.Country;
import com.cognizant.orm_learn.service.CountryService;
import com.cognizant.orm_learn.service.exception.CountryNotFoundException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ConfigurableApplicationContext;

@SpringBootApplication
public class OrmLearnApplication {

    private static final Logger LOGGER =
LoggerFactory.getLogger(OrmLearnApplication.class);

    private static CountryService countryService;

    public static void main(String[] args) {

        ConfigurableApplicationContext context =
SpringApplication.run(OrmLearnApplication.class, args);

        countryService = context.getBean(CountryService.class);

        getAllCountriesTest();
    }
}
```

```

private static void getAllCountriesTest() {

    LOGGER.info("Start");

    try {

        Country country = countryService.findCountryByCode("IN");

        LOGGER.debug("Country:{}", country);

    } catch (CountryNotFoundException e) {

        LOGGER.error("Country not found", e);

    }

    LOGGER.info("End");

}
}

```

The screenshot shows an IDE console window with the following content:

```

<terminated> OrmLearnApplication [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (07-Jul-2026, 1:42:24 pm - 1:42:29 pm)
Database JDBC URL [jdbc:mysql://localhost:3306/ormLearn]
Database driver: MySQL Connector/J
Database dialect: MySQLDialect
Database version: 8.0.37
Default catalog/schema: ormlearn/undefined
Autocommit mode: undefined/unknown
Isolation level: REPEATABLE_READ [default REPEATABLE_READ]
JDBC fetch size: none
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-07-07T13:42:28.501+05:30 INFO 3536 --- [ restartedMain] org.hibernate.orm.core           : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to
2026-07-07T13:42:28.542+05:30 INFO 3536 --- [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-07-07T13:42:28.621+05:30 INFO 3536 --- [ restartedMain] o.s.d.j.r.query.QueryEnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be used.
2026-07-07T13:42:28.912+05:30 INFO 3536 --- [ restartedMain] c.c.orm_learn.OrmLearnApplication : Started OrmLearnApplication in 3.954 seconds (process running for 3.952)
2026-07-07T13:42:28.955+05:30 DEBUG 3536 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Find IN: India
2026-07-07T13:42:28.990+05:30 DEBUG 3536 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:42:29.006+05:30 DEBUG 3536 --- [ restartedMain] org.hibernate.SQL : insert into country (co_name,co_code) values (?,?)
Hibernate: insert into country (co_name,co_code) values (?,?)
Added XX - Testland
2026-07-07T13:42:29.042+05:30 DEBUG 3536 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:42:29.046+05:30 DEBUG 3536 --- [ restartedMain] org.hibernate.SQL : update country set co_name=? where co_code=?
Hibernate: update country set co_name=? where co_code=?
Updated XX
2026-07-07T13:42:29.057+05:30 DEBUG 3536 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:42:29.061+05:30 DEBUG 3536 --- [ restartedMain] org.hibernate.SQL : delete from country where co_code=?
Hibernate: delete from country where co_code=?
Deleted XX
2026-07-07T13:42:29.615+05:30 DEBUG 3536 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) like
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) like
AF - Afghanistan
KZ - Kazakhstan
KG - Kyrgyzstan
PK - Pakistan
SH - Saint Helena, Ascension and Tristan da Cunha
TJ - Tajikistan
TM - Turkmenistan
UZ - Uzbekistan
2026-07-07T13:42:29.620+05:30 INFO 3536 --- [ restartedMain] c.c.orm_learn.OrmLearnApplication : Start
2026-07-07T13:42:29.621+05:30 DEBUG 3536 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:42:29.624+05:30 DEBUG 3536 --- [ restartedMain] c.c.orm_learn.OrmLearnApplication : Country:Country{coCode='IN', coName='India'}
2026-07-07T13:42:29.624+05:30 INFO 3536 --- [ restartedMain] c.c.orm_learn.OrmLearnApplication : End
2026-07-07T13:42:29.630+05:30 INFO 3536 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-07-07T13:42:29.632+05:30 INFO 3536 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown initiated...
2026-07-07T13:42:29.641+05:30 INFO 3536 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown completed.

```

Hands on 7

Add a new country

CountryService.java

java

```
package com.cognizant.orm_learn.service;

import com.cognizant.orm_learn.entity.Country;
import com.cognizant.orm_learn.service.exception.CountryNotFoundException;

import java.util.List;
import java.util.Optional;

public interface CountryService {

    Optional<Country> findByCountryCode(String coCode);

    Country findCountryByCode(String countryCode) throws CountryNotFoundException;

    void addCountry(Country country);

    Country updateCountry(Country country);

    void deleteCountry(String coCode);

    List<Country> findByPartialName(String partialName);
}
```

CountryServiceImpl.java

```
java

package com.cognizant.orm_learn.service;

import com.cognizant.orm_learn.entity.Country;
import com.cognizant.orm_learn.repository.CountryRepository;
```

```
import com.cognizant.orm_learn.service.exception.CountryNotFoundException;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

```
import java.util.List;
import java.util.Optional;
```

```
@Service
```

```
public class CountryServiceImpl implements CountryService {
```

```
    @Autowired
```

```
    private CountryRepository countryRepository;
```

```
    @Override
```

```
    public Optional<Country> findByCountryCode(String coCode) {
```

```
        return countryRepository.findById(coCode);
```

```
    }
```

```
    @Transactional
```

```
    @Override
```

```
    public Country findCountryByCode(String countryCode) throws
    CountryNotFoundException {
```

```
        Optional<Country> result = countryRepository.findById(countryCode);
```

```
        if (!result.isPresent()) {
```

```
            throw new CountryNotFoundException("Country not found for code: " +
    countryCode);
```

```
        }
```

```
        Country country = result.get();  
        return country;  
    }  
}
```

```
@Transactional  
  
@Override  
public void addCountry(Country country) {  
    countryRepository.save(country);  
}  
}
```

```
@Override  
public Country updateCountry(Country country) {  
    return countryRepository.save(country);  
}  
}
```

```
@Override  
public void deleteCountry(String coCode) {  
    countryRepository.deleteById(coCode);  
}  
}
```

```
@Override  
public List<Country> findByPartialName(String partialName) {  
    return countryRepository.findByCoNameContainingIgnoreCase(partialName);  
}  
}  
}
```

OrmLearnApplication.java

java

```
package com.cognizant.orm_learn;

import com.cognizant.orm_learn.entity.Country;
import com.cognizant.orm_learn.service.CountryService;
import com.cognizant.orm_learn.service.exception.CountryNotFoundException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ConfigurableApplicationContext;

@SpringBootApplication
public class OrmLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(OrmLearnApplication.class);

    private static CountryService countryService;

    public static void main(String[] args) {

        ConfigurableApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class, args);

        countryService = context.getBean(CountryService.class);

        getAllCountriesTest();

        testAddCountry();

    }

    private static void getAllCountriesTest() {
```



```

    LOGGER.info("Start");

    try {

        Country country = countryService.findCountryByCode("IN");

        LOGGER.debug("Country:{}, country);

    } catch (CountryNotFoundException e) {

        LOGGER.error("Country not found", e);

    }

    LOGGER.info("End");
}

private static void testAddCountry() {

    LOGGER.info("Start - testAddCountry");

    Country newCountry = new Country("ZZ", "Zedland");

    countryService.addCountry(newCountry);

    try {

        Country fetchedCountry = countryService.findCountryByCode("ZZ");

        LOGGER.debug("Country added and fetched: {}", fetchedCountry);

    } catch (CountryNotFoundException e) {

        LOGGER.error("Country not found after adding", e);

    }

    LOGGER.info("End - testAddCountry");

}
}

```

```

Maximum pool size: undefined/unknown
2026-07-07T13:46:20.962+05:30 INFO 24808 --- [ restartedMain] org.hibernate.orm.core : HH000489: No JTA platform available (set 'hibernate.transaction.jta.platform'
2026-07-07T13:46:21.003+05:30 INFO 24808 --- [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-07-07T13:46:21.114+05:30 INFO 24808 --- [ restartedMain] o.s.d.j.r.query.QueryEnhancerFactory : Hibernate is in classpath; If applicable, HQL parser will be used.
2026-07-07T13:46:21.437+05:30 INFO 24808 --- [ restartedMain] c.c.orm_learn.OrmLearnApplication : Started OrmLearnApplication in 3.684 seconds (process running for 4.109)
2026-07-07T13:46:21.561+05:30 DEBUG 24808 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Find IN: India
2026-07-07T13:46:21.586+05:30 DEBUG 24808 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:46:21.556+05:30 DEBUG 24808 --- [ restartedMain] org.hibernate.SQL : insert into country (co_name,co_code) values (?,?)
Hibernate: insert into country (co_name,co_code) values (?,?)
Added XX - Testland
2026-07-07T13:46:21.586+05:30 DEBUG 24808 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:46:21.590+05:30 DEBUG 24808 --- [ restartedMain] org.hibernate.SQL : update country set co_name=? where co_code=?
Hibernate: update country set co_name=? where co_code=?
Updated XX
2026-07-07T13:46:21.601+05:30 DEBUG 24808 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:46:21.607+05:30 DEBUG 24808 --- [ restartedMain] org.hibernate.SQL : delete from country where co_code=?
Hibernate: delete from country where co_code=?
Deleted XX
2026-07-07T13:46:21.664+05:30 DEBUG 24808 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) l
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) like upper(?) escape '\\'
AF - Afghanistan
KZ - Kazakhstan
KG - Kyrgyzstan
PK - Pakistan
SH - Saint Helena, Ascension and Tristan da Cunha
TJ - Tajikistan
TN - Turkmenistan
UZ - Uzbekistan
2026-07-07T13:46:22.171+05:30 INFO 24808 --- [ restartedMain] c.c.orm_learn.OrmLearnApplication : Start
2026-07-07T13:46:22.172+05:30 DEBUG 24808 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:46:22.174+05:30 DEBUG 24808 --- [ restartedMain] c.c.orm_learn.OrmLearnApplication : Country:Country{coCode='IN', coName='India'}
2026-07-07T13:46:22.174+05:30 INFO 24808 --- [ restartedMain] c.c.orm_learn.OrmLearnApplication : End
2026-07-07T13:46:22.174+05:30 INFO 24808 --- [ restartedMain] c.c.orm_learn.OrmLearnApplication : Start - testAddCountry
2026-07-07T13:46:22.175+05:30 DEBUG 24808 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:46:22.176+05:30 DEBUG 24808 --- [ restartedMain] org.hibernate.SQL : insert into country (co_name,co_code) values (?,?)
Hibernate: insert into country (co_name,co_code) values (?,?)
2026-07-07T13:46:22.189+05:30 DEBUG 24808 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:46:22.191+05:30 DEBUG 24808 --- [ restartedMain] c.c.orm_learn.OrmLearnApplication : Country added and fetched: Country{coCode='ZZ', coName='Zedland'}
2026-07-07T13:46:22.192+05:30 INFO 24808 --- [ restartedMain] c.c.orm_learn.OrmLearnApplication : End - testAddCountry
2026-07-07T13:46:22.197+05:30 INFO 24808 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-07-07T13:46:22.199+05:30 INFO 24808 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown initiated...
2026-07-07T13:46:22.206+05:30 INFO 24808 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown completed.

```

Hands on 1

Write queries on country table using Query Methods

CountryRepository.java

java

package com.cognizant.orm_learn.repository;

import com.cognizant.orm_learn.entity.Country;

import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface CountryRepository extends JpaRepository<Country, String> {

// Query 1: partial name match (already existing)

List<Country> findByCoNameContainingIgnoreCase(String partialName);

```
// Query 2: partial name match, ordered ascending by name
List<Country> findByCoNameContainingIgnoreCaseOrderByCoNameAsc(String
partialName);

// Query 3: countries starting with a given letter
List<Country> findByCoNameStartingWithIgnoreCase(String letter);
}
```

CountryService.java

```
java
package com.cognizant.orm_learn.service;

import com.cognizant.orm_learn.entity.Country;
import com.cognizant.orm_learn.service.exception.CountryNotFoundException;

import java.util.List;
import java.util.Optional;

public interface CountryService {

    Optional<Country> findByCountryCode(String coCode);

    Country findCountryByCode(String countryCode) throws CountryNotFoundException;

    void addCountry(Country country);

    Country updateCountry(Country country);

    void deleteCountry(String coCode);
}
```

```
List<Country> findByPartialName(String partialName);
```

```
List<Country> findByPartialNameOrderByName(String partialName);
```

```
List<Country> findByStartingLetter(String letter);
```

```
}
```

CountryServiceImpl.java

```
java
```

```
package com.cognizant.orm_learn.service;
```

```
import com.cognizant.orm_learn.entity.Country;
```

```
import com.cognizant.orm_learn.repository.CountryRepository;
```

```
import com.cognizant.orm_learn.service.exception.CountryNotFoundException;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Service;
```

```
import org.springframework.transaction.annotation.Transactional;
```

```
import java.util.List;
```

```
import java.util.Optional;
```

```
@Service
```

```
public class CountryServiceImpl implements CountryService {
```

```
    @Autowired
```

```
    private CountryRepository countryRepository;
```

```
    @Override
```

```
public Optional<Country> findByCountryCode(String coCode) {  
    return countryRepository.findById(coCode);  
}
```

@Transactional

@Override

```
public Country findCountryByCode(String countryCode) throws  
CountryNotFoundException {  
    Optional<Country> result = countryRepository.findById(countryCode);  
  
    if (!result.isPresent()) {  
        throw new CountryNotFoundException("Country not found for code: " +  
countryCode);  
    }  
  
    Country country = result.get();  
    return country;  
}
```

@Transactional

@Override

```
public void addCountry(Country country) {  
    countryRepository.save(country);  
}
```

@Override

```
public Country updateCountry(Country country) {  
    return countryRepository.save(country);  
}
```

@Override

```
public void deleteCountry(String coCode) {  
    countryRepository.deleteById(coCode);  
}
```

@Override

```
public List<Country> findByPartialName(String partialName) {  
    return countryRepository.findByCoNameContainingIgnoreCase(partialName);  
}
```

@Override

```
public List<Country> findByPartialNameOrderByName(String partialName) {  
    return  
countryRepository.findByCoNameContainingIgnoreCaseOrderByCoNameAsc(partialN  
ame);  
}
```

@Override

```
public List<Country> findByStartingLetter(String letter) {  
    return countryRepository.findByCoNameStartingWithIgnoreCase(letter);  
}  
}
```

OrmLearnApplication.java

java

package com.cognizant.orm_learn;

import com.cognizant.orm_learn.entity.Country;

import com.cognizant.orm_learn.service.CountryService;

```
import com.cognizant.orm_learn.service.exception.CountryNotFoundException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ConfigurableApplicationContext;
```

```
import java.util.List;
```

```
@SpringBootApplication
```

```
public class OrmLearnApplication {
```

```
    private static final Logger LOGGER =
    LoggerFactory.getLogger(OrmLearnApplication.class);
```

```
    private static CountryService countryService;
```

```
    public static void main(String[] args) {
        ConfigurableApplicationContext context =
        SpringApplication.run(OrmLearnApplication.class, args);
        countryService = context.getBean(CountryService.class);

        getAllCountriesTest();
        testAddCountry();
        testSearchByPartialName();
        testSearchByPartialNameOrdered();
        testSearchByStartingLetter();
    }
```

```
private static void getAllCountriesTest() {  
    LOGGER.info("Start");  
  
    try {  
        Country country = countryService.findCountryByCode("IN");  
  
        LOGGER.debug("Country:{}", country);  
    } catch (CountryNotFoundException e) {  
        LOGGER.error("Country not found", e);  
    }  
  
    LOGGER.info("End");  
}
```

```
private static void testAddCountry() {  
    LOGGER.info("Start - testAddCountry");  
  
    try {  
        countryService.deleteCountry("ZZ");  
    } catch (Exception ignored) {  
    }  
}
```

```
Country newCountry = new Country("ZZ", "Zedland");  
countryService.addCountry(newCountry);
```

```
try {  
    Country fetchedCountry = countryService.findCountryByCode("ZZ");  
  
    LOGGER.debug("Country added and fetched: {}", fetchedCountry);  
} catch (CountryNotFoundException e) {  
    LOGGER.error("Country not found after adding", e);  
}
```



```

        LOGGER.info("End - testAddCountry");
    }

    private static void testSearchByPartialName() {
        LOGGER.info("Start - testSearchByPartialName");

        List<Country> countries = countryService.findByPartialName("ou");

        countries.forEach(c -> LOGGER.debug("{} \t {}", c.getCoCode(), c.getCoName()));

        LOGGER.info("End - testSearchByPartialName");
    }

    private static void testSearchByPartialNameOrdered() {
        LOGGER.info("Start - testSearchByPartialNameOrdered");

        List<Country> countries = countryService.findByPartialNameOrderByName("ou");

        countries.forEach(c -> LOGGER.debug("{} \t {}", c.getCoCode(), c.getCoName()));

        LOGGER.info("End - testSearchByPartialNameOrdered");
    }

    private static void testSearchByStartingLetter() {
        LOGGER.info("Start - testSearchByStartingLetter");

        List<Country> countries = countryService.findByStartingLetter("Z");

        countries.forEach(c -> LOGGER.debug("{} \t {}", c.getCoCode(), c.getCoName()));

        LOGGER.info("End - testSearchByStartingLetter");
    }
}

```

```

Maximum pool size: undefined/unknown
2026-07-07T13:53:12.678+05:30 INFO 5944 --- [ restartedMain] org.hibernate.orm.core : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform
2026-07-07T13:53:12.728+05:30 INFO 5944 --- [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-07-07T13:53:12.868+05:30 INFO 5944 --- [ restartedMain] o.s.d.j.r.query.QueryEntityManagerFactory : Hibernate is in classpath; if applicable, HQL parser will be used.
2026-07-07T13:53:13.243+05:30 INFO 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : Started OmLearnApplication in 4.457 seconds (process running for 4.961)
2026-07-07T13:53:13.338+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Find IN: India
2026-07-07T13:53:13.378+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:53:13.388+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : insert into country (co_name,co_code) values (?,?)
Hibernate: insert into country (co_name,co_code) values (?,?)
Added XX - Testland
2026-07-07T13:53:13.418+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:53:13.435+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : update country set co_name=? where co_code=?
Hibernate: update country set co_name=? where co_code=?
Updated XX
2026-07-07T13:53:13.435+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:53:13.435+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : delete from country where co_code=?
Hibernate: delete from country where co_code=?
Deleted XX
2026-07-07T13:53:14.078+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) like upper(?) escape '\\'
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) like upper(?) escape '\\'
AF - Afghanistan
KZ - Kazakhstan
KG - Kyrgyzstan
PK - Pakistan
SH - Saint Helena, Ascension and Tristan da Cunha
TJ - Tajikistan
TM - Turkmenistan
UZ - Uzbekistan
2026-07-07T13:53:14.086+05:30 INFO 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : Start
2026-07-07T13:53:14.086+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:53:14.086+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : Country:Country{coCode='IN', coName='India'}
2026-07-07T13:53:14.086+05:30 INFO 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : End
2026-07-07T13:53:14.086+05:30 INFO 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : Start - testAddCountry
2026-07-07T13:53:14.086+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:53:14.086+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : delete from country where co_code=?
Hibernate: delete from country where co_code=?
2026-07-07T13:53:14.129+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:53:14.129+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : insert into country (co_name,co_code) values (?,?)
Hibernate: insert into country (co_name,co_code) values (?,?)
2026-07-07T13:53:14.138+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:53:14.086+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : delete from country where co_code=?
Hibernate: delete from country where co_code=?
2026-07-07T13:53:14.129+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:53:14.129+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : insert into country (co_name,co_code) values (?,?)
Hibernate: insert into country (co_name,co_code) values (?,?)
2026-07-07T13:53:14.138+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where c1_0.co_code=?
2026-07-07T13:53:14.138+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : Country added and fetched: Country{coCode='ZZ', coName='Zedland'}
2026-07-07T13:53:14.138+05:30 INFO 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : End - testAddCountry
2026-07-07T13:53:14.138+05:30 INFO 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : Start - testSearchByPartialName
2026-07-07T13:53:14.138+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) like
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) like upper(?) escape '\\'
2026-07-07T13:53:14.145+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : BV Bouvet Island
2026-07-07T13:53:14.145+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : DJ Djibouti
2026-07-07T13:53:14.145+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : TF French Southern Territories
2026-07-07T13:53:14.145+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : GP Guadeloupe
2026-07-07T13:53:14.145+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : LU Luxembourg
2026-07-07T13:53:14.145+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : ZA South Africa
2026-07-07T13:53:14.145+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : GS South Georgia and the South Sandwich Islands
2026-07-07T13:53:14.145+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : SS South Sudan
2026-07-07T13:53:14.145+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : UM United States Minor Outlying Islands
2026-07-07T13:53:14.145+05:30 INFO 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : End - testSearchByPartialName
2026-07-07T13:53:14.145+05:30 INFO 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : Start - testSearchByPartialNameOrdered
2026-07-07T13:53:14.169+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) like
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) like upper(?) escape '\\'
2026-07-07T13:53:14.169+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : BV Bouvet Island
2026-07-07T13:53:14.169+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : DJ Djibouti
2026-07-07T13:53:14.169+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : TF French Southern Territories
2026-07-07T13:53:14.169+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : GP Guadeloupe
2026-07-07T13:53:14.169+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : LU Luxembourg
2026-07-07T13:53:14.169+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : ZA South Africa
2026-07-07T13:53:14.169+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : GS South Georgia and the South Sandwich Islands
2026-07-07T13:53:14.169+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : SS South Sudan
2026-07-07T13:53:14.169+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : UM United States Minor Outlying Islands
2026-07-07T13:53:14.169+05:30 INFO 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : End - testSearchByPartialNameOrdered
2026-07-07T13:53:14.169+05:30 INFO 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : Start - testSearchByStartingLetter
2026-07-07T13:53:14.169+05:30 DEBUG 5944 --- [ restartedMain] org.hibernate.SQL : select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) like
Hibernate: select c1_0.co_code,c1_0.co_name from country c1_0 where upper(c1_0.co_name) like upper(?) escape '\\'
2026-07-07T13:53:14.182+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : ZM Zambia
2026-07-07T13:53:14.182+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : ZW Zimbabwe
2026-07-07T13:53:14.182+05:30 DEBUG 5944 --- [ restartedMain] c.c.orm.learn.OmLearnApplication : ZZ Zedland
2026-07-07T13:53:14.195+05:30 INFO 5944 --- [onShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-07-07T13:53:14.208+05:30 INFO 5944 --- [onShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown initiated...
2026-07-07T13:53:14.208+05:30 INFO 5944 --- [onShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown completed.

```

Query Methods in Spring Data JPA

Introduction

Spring Data JPA provides a powerful feature called **Query Methods**, which allows developers to define database queries simply by declaring method signatures in a repository interface — without writing any SQL or JPQL. Spring parses the method name, understands the intent based on keywords, and automatically generates the underlying query at runtime. This drastically reduces boilerplate code.

Setup

Assume an entity Employee:

```
java
```

```
@Entity
```

```
public class Employee {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

```
    private String department;
```

```
    private Double salary;
```

```
    private LocalDate joiningDate;
```

```
    // getters and setters
```

```
}
```

Repository interface:

```
java
```

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
```

```
    // 1. Search by containing text
```

```
    List<Employee> findByNameContaining(String keyword);
```

```
    // 2. Sorting
```

```
    List<Employee> findByDepartment(String department, Sort sort);
```

```
    // 3. Filter with starting text
```

```
    List<Employee> findByNameStartingWith(String prefix);
```

```
    // 4. Fetch between dates
```

```

List<Employee> findByJoiningDateBetween(LocalDate start, LocalDate end);

// 5. Greater than / Lesser than

List<Employee> findBySalaryGreaterThan(Double salary);

List<Employee> findBySalaryLessThan(Double salary);

// 6. Top N records

List<Employee> findTop3ByOrderBySalaryDesc();
}

```

Explanation of Each Query Method

1. Containing (LIKE %keyword%)

findByNamesContaining("John") generates a query equivalent to WHERE name LIKE %John%. Useful for partial text search.

2. Sorting

findByDepartment("IT", Sort.by("salary").descending()) fetches all employees in the IT department sorted by salary in descending order. The Sort object is passed as a parameter.

3. Starting With

findByNamesStartingWith("A") returns all employees whose name starts with "A" — equivalent to WHERE name LIKE 'A%'.

4. Between Dates

findByJoiningDateBetween(startDate, endDate) returns employees who joined within a specific date range — equivalent to WHERE joiningDate BETWEEN start AND end.

5. Greater Than / Less Than

findBySalaryGreaterThan(50000.0) and findBySalaryLessThan(30000.0) filter records based on numeric comparison — equivalent to WHERE salary > 50000 or WHERE salary < 30000.

6. Top / First

findTop3ByOrderBySalaryDesc() returns only the top 3 highest-paid employees. Keywords Top or First combined with OrderBy limit and sort results in a single method.

Sample Usage in Service Layer

```
java
```

```

@Service
public class EmployeeService {

    @Autowired

    private EmployeeRepository employeeRepository;


    public List<Employee> searchByName(String keyword) {

        return employeeRepository.findByNameContaining(keyword);

    }


    public List<Employee> getTopEarners() {

        return employeeRepository.findTop3ByOrderBySalaryDesc();

    }

}

```

Conclusion

Spring Data JPA's Query Methods feature eliminates the need to write manual SQL/JPQL for common query patterns. By following naming conventions (findBy, Containing, StartingWith, Between, GreaterThan, LessThan, Top, OrderBy), Spring automatically derives and executes the correct query, making development faster, cleaner, and less error-prone.

Object-Relational (O/R) Mapping in JPA/Hibernate

Introduction

Object-Relational Mapping (ORM) is a technique that maps Java objects to relational database tables. JPA (Java Persistence API), implemented by Hibernate, provides annotations to define relationships between entities such as One-to-Many, Many-to-One, and Many-to-Many. These annotations describe how tables relate to each other and how data should be fetched.

1. @ManyToOne and @JoinColumn

Represents a relationship where many records of one entity relate to a single record of another. For example, many Employee records belong to one Department.

java

@Entity

```
public class Employee {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

```
    @ManyToOne(fetch = FetchType.LAZY)
```

```
    @JoinColumn(name = "department_id") // foreign key column in Employee table
```

```
    private Department department;
```

```
}
```

@JoinColumn specifies the foreign key column name in the owning table (Employee) that references the primary key of the Department table.

2. @OneToMany and mappedBy

Represents the inverse side of the relationship — one Department has many Employees.

```
java
```

@Entity

```
public class Department {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String deptName;
```

```
    @OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
```

```
    private List<Employee> employees;
```

```
}
```

mappedBy = "department" tells Hibernate that the Employee entity already owns the relationship (via the department field), so Department should not create a separate foreign key or join table — it simply reads the existing mapping.

3. FetchType.EAGER vs FetchType.LAZY

- **EAGER:** Related data is fetched immediately along with the parent entity, even if not accessed. Example: @ManyToOne(fetch = FetchType.EAGER) — when an Employee is loaded, its Department is loaded too, right away.
- **LAZY:** Related data is fetched only when explicitly accessed (on-demand), improving performance by avoiding unnecessary joins.

Fetch Type	Default For	Behavior
EAGER	@ManyToOne, @OneToOne	Loads related entity immediately
LAZY	@OneToMany, @ManyToMany	Loads related entity only when accessed

Best practice: Use LAZY wherever possible to avoid performance issues, and fetch relations explicitly when needed using JPQL JOIN FETCH.

4. @ManyToMany and @JoinTable

Represents a relationship where many records of one entity relate to many records of another. Example: Students and Courses.

```
java
```

```
@Entity
```

```
public class Student {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

```
    @ManyToMany(fetch = FetchType.LAZY)
```

```
    @JoinTable(
```

```
        name = "student_course",           // join table name
```

```
        joinColumns = @JoinColumn(name = "student_id"), // FK referring to Student
```

```

        inverseJoinColumns = @JoinColumn(name = "course_id") // FK referring to Course
    )

    private List<Course> courses;
}

```

@Entity

```
public class Course {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String courseName;
```

```
    @ManyToMany(mappedBy = "courses")
```

```
    private List<Student> students;
```

```
}
```

@JoinTable creates a separate junction/mapping table (student_course) containing two foreign keys (student_id and course_id) since a many-to-many relationship cannot be represented with a single foreign key.

Conclusion

O/R Mapping annotations in JPA allow Java developers to model complex real-world relationships (one-to-many, many-to-one, many-to-many) directly through entity classes without manually writing SQL joins. Proper use of @JoinColumn, mappedBy, and @JoinTable, along with appropriate fetch strategies, ensures efficient and maintainable database interactions.

Hibernate Query Language (HQL) and Native Query

Introduction

While Query Methods handle simple queries through method naming conventions, more complex queries require writing actual query language. Spring Data JPA supports this through the @Query annotation, allowing developers to write **HQL/JPQL** (object-oriented queries) or **Native SQL** (direct database queries) inside repository interfaces.

What is HQL?

HQL (Hibernate Query Language) is Hibernate's own object-oriented query language. Instead of operating on tables and columns, HQL operates on **entities and their properties** (Java objects and fields). It is database-independent, meaning the same HQL query works across MySQL, PostgreSQL, Oracle, etc., because Hibernate translates it into the appropriate SQL dialect at runtime.

Example:

java

```
@Query("SELECT e FROM Employee e WHERE e.department = :dept")
```

```
List<Employee> findEmployeesByDept(@Param("dept") String dept);
```

Here, Employee refers to the **entity class name**, not the table name, and e.department refers to the **field name** in the Java class.

What is JPQL?

JPQL (Java Persistence Query Language) is the standard query language defined by the JPA specification (not tied to Hibernate specifically). Its syntax is almost identical to HQL since Hibernate is the most common JPA implementation and HQL is a superset of JPQL.

java

```
@Query("SELECT e FROM Employee e WHERE e.salary > :minSalary")
```

```
List<Employee> findHighEarners(@Param("minSalary") Double minSalary);
```

Comparison: HQL vs JPQL

Aspect	HQL	JPQL
Defined by	Hibernate (vendor-specific)	JPA Specification (standard)
Portability	Tied to Hibernate	Works with any JPA provider (Hibernate, EclipseLink, OpenJPA)
Feature set	Superset — has extra Hibernate-only features (e.g., WITH, additional functions)	Subset — only standard features defined by JPA spec
Syntax	Almost identical to JPQL	Almost identical to HQL
Use case	When using Hibernate-specific features	When database/provider portability is important

In short: All JPQL queries are valid HQL, but not all HQL queries are valid JPQL, since HQL provides some additional non-standard extensions.

The @Query Annotation

@Query is used in a repository interface to explicitly define a query instead of relying on method-name derivation. It can accept either HQL/JPQL (default) or Native SQL (with nativeQuery = true).

java

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
```

```
    @Query("SELECT e FROM Employee e WHERE e.name LIKE %:keyword%")
```

```
    List<Employee> searchByName(@Param("keyword") String keyword);
```

```
}
```

@Param binds the method parameter to the named parameter (:keyword) inside the query string.

HQL FETCH Keyword

The FETCH keyword is used in HQL/JPQL to eagerly load associated entities in a **single query using a JOIN**, avoiding the N+1 select problem (where Hibernate fires one query per related row).

java

```
@Query("SELECT d FROM Department d JOIN FETCH d.employees WHERE d.id = :id")
```

```
Department findDeptWithEmployees(@Param("id") Long id);
```

Without FETCH, a normal JOIN would only fetch Department fields, and Hibernate would trigger additional lazy queries when employees is accessed. JOIN FETCH retrieves both Department and its employees in one go, improving performance.

Aggregate Functions in HQL

HQL supports standard SQL aggregate functions for computing summary values:

java

```
// COUNT - total number of employees
```

```
@Query("SELECT COUNT(e) FROM Employee e")
```

```
Long countEmployees();
```

```
// SUM - total salary expenditure
```

```
@Query("SELECT SUM(e.salary) FROM Employee e")
```

```
Double totalSalary();
```

```
// AVG - average salary
```

```
@Query("SELECT AVG(e.salary) FROM Employee e WHERE e.department = :dept")
```

```
Double averageSalaryByDept(@Param("dept") String dept);
```

```
// MAX and MIN
```

```
@Query("SELECT MAX(e.salary) FROM Employee e")
```

```
Double maxSalary();
```

```
@Query("SELECT MIN(e.salary) FROM Employee e")
```

```
Double minSalary();
```

```
// GROUP BY with aggregate
```

```
@Query("SELECT e.department, COUNT(e) FROM Employee e GROUP BY  
e.department")
```

```
List<Object[]> countEmployeesByDept();
```

These functions work exactly like their SQL counterparts but operate on entity fields rather than table columns.

Native Query

A **Native Query** is a raw SQL query written exactly as the underlying database engine understands it — including actual table names and column names (not entity/field names). It is used when a query is too complex or database-specific to express in HQL/JPQL (e.g., using vendor-specific functions, stored procedures, or complex joins).

```
java
```

```
@Query(value = "SELECT * FROM employee WHERE salary > ?1", nativeQuery = true)
```

```
List<Employee> findHighEarnersNative(Double salary);
```

The nativeQuery Attribute

The nativeQuery attribute inside @Query tells Spring Data JPA whether the given query string should be interpreted as HQL/JPQL (false, the default) or as raw SQL (true).

```
java
```

```
@Query(value = "SELECT * FROM employee e WHERE e.department_id = :deptId  
ORDER BY e.salary DESC LIMIT 3",
```

```
    nativeQuery = true)
```

```
List<Employee> findTop3ByDeptNative(@Param("deptId") Long deptId);
```

Since this is a native query, employee, department_id, and salary refer to actual **database table/column names**, not Java entity/field names.

HQL vs Native Query — Quick Comparison

Aspect	HQL/JPQL	Native Query
Operates on	Entities & fields	Tables & columns
Database independence	Yes	No (tied to specific DB syntax)
Complex DB-specific features	Limited	Fully supported
Readability/OOP alignment	High	Low (plain SQL)
nativeQuery attribute	false (default)	true

Sample Complete Repository

```
java
```

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
```

```
    @Query("SELECT e FROM Employee e WHERE e.department = :dept")
```

```
    List<Employee> findByDeptHQL(@Param("dept") String dept);
```

```
    @Query("SELECT d FROM Department d JOIN FETCH d.employees WHERE d.id = :id")
```

```
    Department findWithEmployees(@Param("id") Long id);
```

```
@Query("SELECT AVG(e.salary) FROM Employee e")
```

```
Double getAverageSalary();
```

```
@Query(value = "SELECT * FROM employee WHERE department_id = :deptId",  
nativeQuery = true)
```

```
List<Employee> findByDeptNative(@Param("deptId") Long deptId);
```

```
}
```

Conclusion

HQL and JPQL provide an object-oriented way to query data using entity names and fields, making queries portable and easier to maintain, while the FETCH keyword and aggregate functions add power for performance optimization and data summarization. Native Queries, enabled via `nativeQuery = true` in `@Query`, allow developers to fall back to raw SQL when database-specific operations are required. Together, these tools give Spring Data JPA developers full flexibility — from simple derived queries to complex custom SQL.